

Technologies : Tkinter, SQLite, MVC

1. Analyse des besoins (modélisation des tâches)

L'objectif de ce TP est de concevoir et développer une application permettant à un utilisateur de gérer ses contacts personnels ou professionnels.

L'application doit permettre la **création**, la **recherche**, la **modification**, la **suppression** et l'**affichage intégral** de contacts dans une base SQLite.

Tâche 1 – Ajouter un contact

L'utilisateur doit pouvoir saisir les informations suivantes :

- ✓ **Nom** (obligatoire)
- ✓ **Prénom** (obligatoire)
- ✓ **Téléphone**
- ✓ **Email**
- ✓ **Adresse postale** (optionnelle)

À la validation :

- ✓ Le contact est stocké dans la base.
- ✓ Un message confirme l'enregistrement.
- ✓ La liste des contacts s'actualise automatiquement.

Tâche 2 – Rechercher un contact

L'utilisateur peut saisir un critère de recherche :

- ✓ Par nom
- ✓ Ou par téléphone
- ✓ Ou par email

La recherche doit utiliser des commandes SQL permettant la correspondance partielle (ex : LIKE).

Tâche 3 – Modifier un contact

- ✓ L'utilisateur sélectionne un contact dans la liste.
- ✓ Les champs du formulaire se remplissent automatiquement.
- ✓ Après modification, il clique sur « Modifier ».
- ✓ Les informations sont mises à jour dans la base.

Tâche 4 – Supprimer un contact

- ✓ L'utilisateur sélectionne un contact.
- ✓ Une boîte de confirmation s'affiche.
- ✓ Le contact est supprimé s'il confirme.

Tâche 5 – Afficher tous les contacts

Affichage complet du carnet d'adresses trié alphabétiquement par nom.

Tâche 6 – Quitter l'application

2. IHM abstraite

L'IHM abstraite décrit la structure fonctionnelle de l'interface, indépendamment de la mise en forme.

Zones de saisie

- ✓ zone_nom
- ✓ zone_prenom
- ✓ zone_telephone
- ✓ zone_email
- ✓ zone_adresse

Ces champs doivent permettre de remplir ou modifier les informations d'un contact.

Boutons d'action

- ✓ bouton_ajouter
- ✓ bouton_rechercher
- ✓ bouton_modifier
- ✓ bouton_supprimer
- ✓ bouton_afficher_tous
- ✓ bouton_quitter

Zone d'affichage

Une table abstraite représentant les contacts sous forme de colonnes :

1. Nom
2. Prénom
3. Téléphone
4. Email
5. Adresse

Sélection

Sélection d'une ligne dans la liste pour :

- ✓ Modifier
- ✓ Supprimer

Cette sélection doit permettre de récupérer l'identifiant unique du contact.

Boîtes de dialogue abstraites

- ✓ erreur_saisie : email invalide, champ obligatoire vide
- ✓ confirmation_suppression : demande avant effacement
- ✓ validation_action : contact ajouté/modifié/supprimé

3. IHM concrète (Tkinter)

Fenêtre principale

- ✓ Titre : « Carnet d'adresses »
- ✓ Frame supérieure : formulaire de saisie
- ✓ Frame centrale : boutons d'actions
- ✓ Frame inférieure : liste des contacts (Treeview)

Formulaire

Champ	Widget Tkinter
Nom	Label + Entry
Prénom	Label + Entry
Téléphone	Label + Entry
Email	Label + Entry
Adresse	Label + Text (3 lignes)

L'email doit être validé dans le contrôleur :

- ✓ Vérifier la présence de @ et d'un domaine valide.

Zone d'action

Boutons disposés horizontalement :

- ✓ Ajouter
- ✓ Rechercher
- ✓ Modifier
- ✓ Supprimer
- ✓ Afficher tous
- ✓ Quitter

Zone d'affichage (Frame inférieure)

Utilisation d'un Treeview avec scrollbar.

Colonnes visibles :

- ✓ Nom
- ✓ Prénom
- ✓ Téléphone
- ✓ Email
- ✓ Adresse

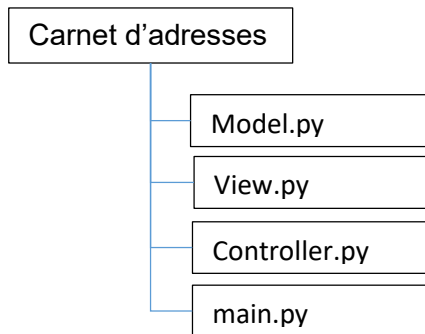
Le double-clic peut être utilisé pour charger automatiquement les informations dans les champs.

Boîtes de dialogue concrètes

- ✓ `messagebox.showerror("Erreur", "Email invalide.")`
- ✓ `messagebox.askyesno("Confirmation", "Supprimer ce contact ?")`
- ✓ `messagebox.showinfo("Succès", "Contact ajouté.")`

4. Architecture MVC

Le programme doit être structuré en quatre fichiers :



4.1 Modèle (model.py)

Fonctions du modèle

- ✓ ajouter_contact()
- ✓ rechercher_contact()
- ✓ modifier_contact()
- ✓ supprimer_contact()
- ✓ afficher_tous()

Le modèle ne gère aucune interface graphique.

4.2 Vue (view.py)

La vue contient :

- ✓ la fenêtre Tkinter
- ✓ le formulaire
- ✓ le Treeview
- ✓ les méthodes pour afficher les résultats

Fonctions importantes

- ✓ get_inputs()
- ✓ set_inputs(contact)
- ✓ afficher_liste(liste_contacts)
- ✓ message_info(), message_erreur(), message_confirmation()

La vue ne contient jamais de requête SQL.

4.3 Contrôleur (controller.py)

Le contrôleur :

- ✓ récupère les données de la vue
- ✓ les valide
- ✓ appelle le modèle
- ✓ met à jour la vue

4.4 main.py

Initialise les trois composants et lance Tkinter.

5. Travail demandé

1. Modélisation des tâches

- ✓ Décrire chaque cas d'utilisation avec les flux de travail.

2. IHM abstraite

- ✓ Identifier les zones, boutons, interactions.

3. IHM concrète

- ✓ Réaliser une maquette (Figma, Pencil, Draw.io...).

4. Développement Python

- ✓ Implémenter le modèle SQLite
- ✓ Construire l'interface Tkinter
- ✓ Programmer le contrôleur
- ✓ Structurer selon MVC

5. Livrables

- ✓ Code source complet
- ✓ Fichier SQLite généré
- ✓ Schémas (IHM abstraite, concrète)
- ✓ Rapport (10 pages maximum)